Local Reconnaissance and Attack Surface Map

Ethical Hacking | Beginner-friendly lab assessment

Report date: *13 October 2026*
Target: *OWASP Juice Shop — http://localhost:3000*

AUTHORIZED SCOPE
Only the local OWASP Juice Shop lab running in Docker, bound to 127.0.0.1:3000.

1. Executive Summary

This report documents reconnaissance performed against a locally hosted OWASP Juice Shop instance as part of a beginner-friendly ethical hacking exercise. The observed target was http://localhost:3000, with Docker configured to bind the host port to 127.0.0.1:3000, keeping the service local to the machine.

The work identified the web application, several JavaScript assets, HTTP response headers, robots.txt behavior, Swagger UI documentation, and a documented order-creation API operation. The observations support a preliminary attack-surface inventory and follow-up test plan. No vulnerability was confirmed during the checks described here.

2. Scope and Lab Safety

Authorized target: OWASP Juice Shop running locally in Docker.

Target URL: http://localhost:3000

Observed host binding: 127.0.0.1:3000 -> container port 3000/tcp.

Testing boundaries: checks were limited to the local lab. The work used HTTP requests and inspection of locally served documentation and JavaScript assets. No destructive tests, credential attacks, or order-creation requests were performed.

Assessment limitation: this is a preliminary reconnaissance report based on the command output captured during the session. It is not a full penetration test or proof of exploitability.

3. Methodology

• Confirmed the local web application was responding and reviewed Docker logs for service startup.

• Sent HTTP header and OPTIONS requests to observe response headers and advertised methods.

• Requested robots.txt and sitemap.xml and recorded their actual responses.

• Saved the homepage HTML and identified referenced JavaScript assets.

• Opened Swagger UI and inspected the embedded OpenAPI 3.0.0 specification in swagger-ui-init.js.

• Compared GET /orders with a deliberately nonexistent path to understand frontend fallback behavior.

• Recorded observations separately from risk hypotheses and unverified questions.

4. Attack Surface Inventory

Target service: HTTP web application on TCP port 3000. The application identified itself in HTML as OWASP Juice Shop. The exact server/framework version was not established from the collected evidence.

Frontend assets observed: rolldown-runtime-BoHGiXSq.js, chunk-DBPdFzgj.js, chunk-Cv2YgWUq.js, chunk-1qQQGal7.js, polyfills.js, scripts.js, and main.js. HEAD requests to /main.js and /scripts.js returned HTTP 200 with application/javascript content type. Observed content lengths: main.js 1,242,633 bytes; scripts.js 20,700 bytes.

Documentation route: /api-docs returned HTTP 301 with Location: /api-docs/, and following the redirect returned HTTP 200 and a Swagger UI page.

Documented API route: POST /orders, operationId createCustomerOrder, description “Create new customer order.” It accepts an application/json request body referencing the Order schema and documents a 200 response.

Order schema inputs: cid is a required string; orderLines references an array of OrderLine items; orderLinesData is a string in a customer-specific format. Each OrderLine requires integer productId and integer quantity with a documented minimum of 1; customerReference is an optional string.

Authentication documentation: the OpenAPI document defines bearerAuth as HTTP Bearer with JWT format and applies it as a global security requirement. Runtime enforcement for POST /orders was not verified.

Visible frontend inputs: a complete manual inventory of interactive forms and fields was not performed. The request fields above are from API schema documentation, not proof that each field appears as a visible browser form.

5. HTTP Headers and Route Observations

GET / (HEAD request) returned HTTP 200 OK with Content-Type: text/html; charset=UTF-8 and Content-Length: 9984.

Observed response headers included Access-Control-Allow-Origin: *, X-Content-Type-Options: nosniff, X-Frame-Options: SAMEORIGIN, Feature-Policy: payment 'self', X-Recruiting: /#/jobs, Cache-Control: public, max-age=0, and ETag: W/"2700-1a13620c76e".

OPTIONS / returned HTTP 204 No Content and advertised Access-Control-Allow-Methods: GET,HEAD,PUT,PATCH,POST,DELETE. This header alone does not prove that each method is enabled or functional on every route.

GET /robots.txt returned HTTP 200 and body: User-agent: * / Disallow: /ftp. A robots.txt disallow directive is a crawler instruction, not access control and not proof that /ftp is exposed or vulnerable.

GET /sitemap.xml returned HTTP 200 but Content-Type: text/html; charset=UTF-8 and the Juice Shop homepage HTML rather than an XML sitemap. This may be frontend fallback behavior; the intended sitemap route was not confirmed.

GET /orders returned HTTP 200 with HTML content type and homepage HTML. GET /this-path-should-not-exist also returned HTTP 200 with HTML content type. These results are consistent with frontend fallback routing, but exact response-body equality was not measured with a hash or byte comparison.

Swagger UI and the embedded OpenAPI document are available at /api-docs/ and /api-docs/swagger-ui-init.js respectively.

6. Findings and Risk Hypotheses

F-01 — API documentation exposed within the lab (Informational). Evidence: /api-docs/ returned Swagger UI and an embedded OpenAPI 3.0.0 document. Hypothesis: documentation may reveal route names, schemas, and authentication expectations. In this deliberately vulnerable training application, documentation exposure is an observation, not a confirmed flaw.

F-02 — Order creation operation documented (Review priority: High). Evidence: POST /orders accepts JSON with customer/order fields and is described as creating an order. Hypothesis: if authentication, authorization, validation, or customer ownership checks are missing or weak, order integrity could be affected. None of these weaknesses was confirmed.

F-03 — JWT bearer scheme declared globally (Review priority: High). Evidence: bearerAuth is defined as HTTP Bearer/JWT and appears under the OpenAPI global security requirement. Hypothesis: runtime enforcement should be compared with the documented requirement. The GET /orders result is not a valid authentication test for the documented POST operation because it returned the frontend HTML fallback.

F-04 — Wildcard CORS origin observed (Review priority: Medium, context-dependent). Evidence: Access-Control-Allow-Origin: * appeared in observed responses. Hypothesis: review which resources and endpoints return this header and whether credentialed cross-origin access is possible. A wildcard origin alone is not necessarily a vulnerability, particularly for public, non-credentialed resources.

F-05 — Multiple HTTP methods advertised in OPTIONS response (Informational/Review). Evidence: GET, HEAD, PUT, PATCH, POST, and DELETE were listed in Access-Control-Allow-Methods for OPTIONS /. Hypothesis: verify method handling per API route. This header is not proof that state-changing methods work on every path.

F-06 — Frontend fallback returns HTTP 200 for unrecognized paths (Informational). Evidence: /orders and /this-path-should-not-exist returned HTML with status 200. Hypothesis: status-only route discovery may produce false positives. This behavior is not itself a vulnerability.

F-07 — robots.txt disallows /ftp (Informational). Evidence: robots.txt contains Disallow: /ftp. Hypothesis: if the path is relevant to the app, review it through authorized, non-destructive checks. The directive does not prove that the path is accessible or sensitive.

F-08 — Security-related headers observed (Positive observation). Evidence: X-Content-Type-Options: nosniff and X-Frame-Options: SAMEORIGIN were present. These are protective signals, though they do not establish the overall security posture. A broader review of Content-Security-Policy and other headers was not completed.

7. Prioritized Next Tests

Priority 1 — Confirm route and authentication behavior safely. In the local lab, use the API documentation to understand the expected method and request format; compare responses for an unauthenticated request without submitting an order. Avoid sending POST requests that create data until the test plan explicitly calls for it.

Priority 2 — Review authorization and customer ownership assumptions in the order schema. Determine what cid represents and whether it is tied to an authenticated customer. Do not assume a vulnerability from the schema alone.

Priority 3 — Review input validation expectations for productId, quantity, customerReference, and orderLinesData. The schema documents quantity minimum 1 but does not by itself prove server-side validation.

Priority 4 — Characterize CORS behavior by resource and endpoint. Record Origin, credentials, and preflight behavior using harmless requests. Do not infer exploitability from Access-Control-Allow-Origin: * alone.

Priority 5 — Enumerate additional API routes and visible UI inputs through the documented specification and browser interface, recording method, path, authentication expectations, input fields, and response type.

Priority 6 — Record application/server version information only where a reliable banner, package metadata, or documentation explicitly provides it; do not guess versions from asset filenames.

8. Limitations

No confirmed vulnerability was found in the observations documented here.

Server/framework versions were not reliably identified.

A full inventory of browser-visible inputs was not completed.

Authentication enforcement for POST /orders was not tested directly.

No order was created, and no destructive or state-changing endpoint was invoked.

The response to /sitemap.xml and the unknown-path response may reflect frontend fallback behavior; this interpretation remains provisional.

9. Conclusion

The local lab’s initial attack surface includes the web application on port 3000, its JavaScript assets, Swagger UI/OpenAPI documentation, and the documented POST /orders operation. Useful defensive controls were observed in response headers, while wildcard CORS, route behavior, and the order API’s authentication/authorization expectations warrant context-aware follow-up.

The report distinguishes direct observations from hypotheses. The next stage should verify behavior in the authorized local lab, collect reproducible evidence, and update each finding with a status such as Confirmed, Not Reproducible, or Inconclusive.

Appendix A — Reproducible Commands Used

```
curl.exe -I http://localhost:3000

curl.exe -i http://localhost:3000/robots.txt

curl.exe -i http://localhost:3000/sitemap.xml

curl.exe -i -X OPTIONS http://localhost:3000/

curl.exe -s http://localhost:3000/ -o index.html

Select-String -Path .\index.html -Pattern 'src="[^"]+\.js"|href="[^"]+\.js"'

curl.exe -i -L http://localhost:3000/api-docs

curl.exe -s http://localhost:3000/api-docs/swagger-ui-init.js -o swagger-init.js

Select-String -Path .\swagger-init.js -Pattern '"/[^\"]+"\s*:|"(get|post|put|patch|delete)"\s*:' -CaseSensitive:$false
```

Get-Content .\swagger-init.js | Select-Object -Skip 25 -First 65

Get-Content .\swagger-init.js | Select-Object -Skip 75 -First 100

curl.exe -i http://localhost:3000/orders

curl.exe -s -o NUL -w "HTTP status: %{http_code} | Content type: %{content_type}\n"
http://localhost:3000/this-path-should-not-exist

Appendix B — Submission Checklist

■ Confirm the lab remains bound to localhost and is not exposed publicly.

■ Add screenshots of the local app, Swagger UI, and selected command outputs if the assignment reviewer expects visual evidence.

■ Review the report for any personal information before sharing.

■ Upload this report and supporting evidence to one repository, folder, or project page.

■ Set the share permission to public/view-only as appropriate and verify the link opens in a private/incognito window.

■ Submit one shareable link to the assignment platform.